

DGraph 金融反欺诈图算法开发报告

1 模型选择与背景

1.1 为什么选择 CARE-GNN

在金融反欺诈场景中，传统的 GNN 模型（如 GCN、GAT、GraphSAGE）面临一个核心问题：**欺诈者的伪装行为**。欺诈节点会通过建立大量与正常用户的连接来“稀释”自身的异常特征，这种现象被称为**伪装攻击（Camouflage Attack）**。

CARE-GNN (Camouflage-Resistant GNN) 是一种专门针对欺诈检测场景设计的图神经网络。该模型的核心优势在于：它通过相似度计算模块识别并过滤掉欺诈者用于伪装的虚假连接，避免正常用户的特征“污染”欺诈节点的表示；同时采用强化学习机制动态调整邻居选择阈值，能够自适应地应对不同节点的邻居分布差异；此外，模型支持多关系异构图，通过分关系聚合和跨关系注意力融合充分利用 11 种边类型的信息；最后，引入标签相似度辅助任务增强监督信号，缓解欺诈样本稀少导致的信号稀疏问题。

1.2 CARE-GNN 核心思想

CARE-GNN 的核心是“先过滤，再聚合”：

传统GNN： 节点 → 聚合所有邻居 → 分类

CARE-GNN： 节点 → 计算邻居相似度 → RL选择可信邻居 → 聚合 → 分类

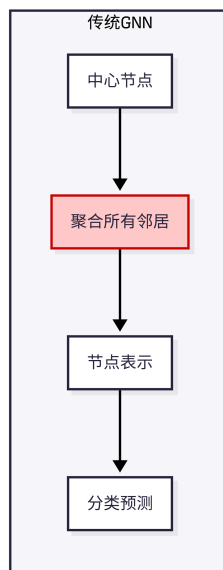
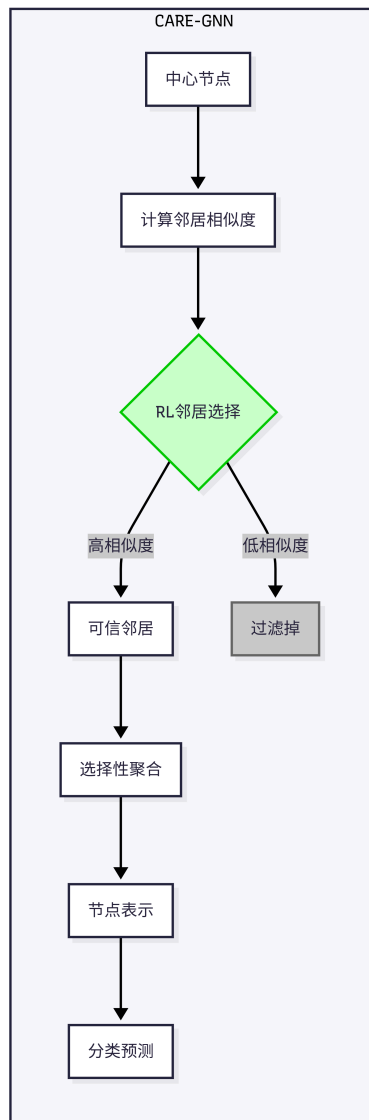


Figure 1: CARE-GNN 架构示意图

2 模型架构与关键代码

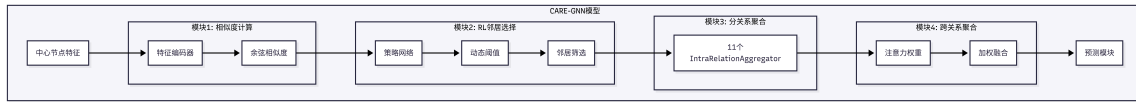


Figure 2: CARE-GNN 架构示意图

2.1 整体架构

输入特征 → 相似度计算 → RL邻居选择 → 分关系聚合 → 跨关系聚合 → [2-hop扩展] → 预测

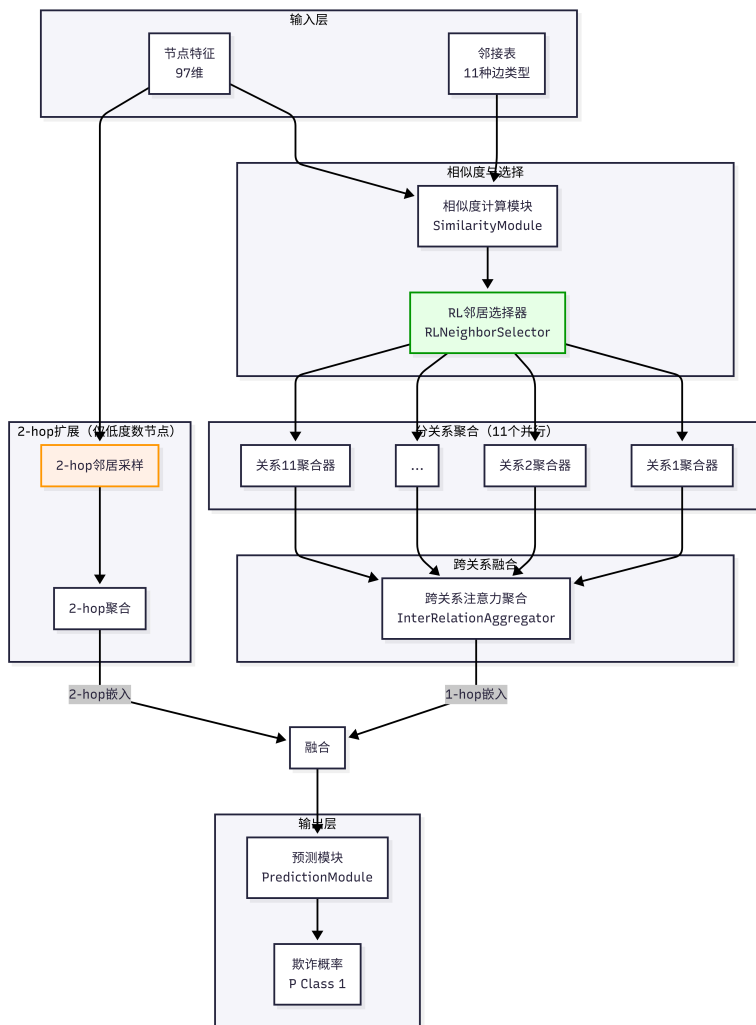


Figure 3: CARE-GNN 架构示意图

2.2 核心模块实现

2.2.1 模块 1：相似度计算模块

作用：计算中心节点与邻居节点的特征相似度，用于识别伪装邻居。

关键点：使用余弦相似度而非内积，避免特征尺度影响。

```
# === 代码位置: care_gnn_modules.py ===
class SimilarityModule(nn.Module):
    """特征相似度计算模块"""
```

```

def __init__(self, feature_dim, embed_dim):
    super(SimilarityModule, self).__init__()
    # 将原始特征映射到嵌入空间
    self.feature_encoder = nn.Sequential(
        nn.Linear(feature_dim, embed_dim),
        nn.ReLU(),
        nn.Linear(embed_dim, embed_dim)
    )

def forward(self, center_features, neighbor_features):
    """
    计算中心节点与邻居的相似度
    center_features: (batch_size, feature_dim)
    neighbor_features: (batch_size, num_neighbors, feature_dim)
    """
    # 编码特征
    center_emb = self.feature_encoder(center_features)
    neighbor_emb = self.feature_encoder(neighbor_features)

    # 计算余弦相似度
    center_emb = F.normalize(center_emb, p=2, dim=1).unsqueeze(1)
    neighbor_emb = F.normalize(neighbor_emb, p=2, dim=2)
    similarity = (center_emb * neighbor_emb).sum(dim=2)

    return similarity

```

2.2.2 模块 2：强化学习邻居选择器

作用：动态学习最优的邻居过滤阈值，而非使用固定阈值。

关键点：阈值不是固定的超参数，而是由策略网络根据节点特征动态生成。

```

# === 代码位置: care_gnn_modules.py ===
class RLNeighborSelector(nn.Module):
    """基于强化学习的邻居选择器"""

    def __init__(self, embed_dim, step_size=0.02):
        super(RLNeighborSelector, self).__init__()
        self.step_size = step_size

        # 策略网络：根据节点嵌入决定阈值
        self.policy_net = nn.Sequential(
            nn.Linear(embed_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
            nn.Sigmoid() # 输出阈值 [0, 1]
        )

    def select_neighbors(self, similarity_scores, center_emb, training=True):
        # 策略网络预测阈值
        threshold = self.policy_net(center_emb)

```

```

# 训练时添加探索噪声
if training:
    noise = torch.randn_like(threshold) * self.step_size
    threshold = torch.clamp(threshold + noise, 0.0, 1.0)

# 选择高于阈值的邻居
selected_mask = (similarity_scores > threshold).float()
return selected_mask, threshold

```

2.2.3 模块 3：分关系聚合

作用：对 11 种不同类型的边分别进行邻居聚合，保留异构信息。

```

# === 代码位置: care_gnn_model.py ===

# __init__ 中: 为每种关系类型创建独立的聚合器
self.intra_aggregators = nn.ModuleList([
    IntraRelationAggregator(feature_dim, embed_dim)
    for _ in range(num_relations) # 11种边类型
])

# forward 中: 分关系处理
for rel_type in range(1, self.num_relations + 1):
    # 采样该关系类型的邻居
    neighbor_features, neighbor_mask = self.sample_neighbors(
        nodes, adj_lists, rel_type, features
    )

    # 计算相似度并选择邻居
    similarity_scores = self.similarity_module(center_features, neighbor_features)
    selected_mask, _ = self.neighbor_selector.select_neighbors(...)

    # 使用该关系的专属聚合器
    relation_emb = self.intra_aggregators[rel_type - 1](
        center_features, neighbor_features, selected_mask
    )
    relation_embeddings.append(relation_emb)

```

2.2.4 模块 4：跨关系注意力聚合

作用：将 11 种关系的嵌入融合为最终表示，使用注意力机制学习各关系的重要性。

```

# === 代码位置: care_gnn_modules.py ===

class InterRelationAggregator(nn.Module):
    """跨关系聚合器 - 注意力机制"""

    def __init__(self, embed_dim, num_relations, agg_type='attention'):
        super().__init__()
        if agg_type == 'attention':
            self.attention = nn.Sequential(
                nn.Linear(embed_dim, 32),
                nn.Tanh(),

```

```

        nn.Linear(32, 1)
    )

    def forward(self, relation_embeddings):
        """relation_embeddings: List of (batch_size, embed_dim)"""
        stacked = torch.stack(relation_embeddings, dim=1) # (B, R, D)

        # 计算注意力权重
        attn_scores = self.attention(stacked).squeeze(-1) # (B, R)
        attn_weights = F.softmax(attn_scores, dim=1).unsqueeze(-1)

        # 加权聚合
        output = (stacked * attn_weights).sum(dim=1) # (B, D)
        return output

```

3 针对比赛的适配与增强

3.1 增强 1：自适应采样 + 2-hop 邻居扩展

问题：数据分析发现 70%+节点度数 ≤ 2 ，传统 1-hop 聚合获取的信息极其有限。

解决方案：引入自适应采样和 2-hop 邻居扩展机制。

```

# === 代码位置: care_gnn_model.py ===

# 自适应邻居采样数配置 (按度数层级)
self.adaptive_max_neighbors = {
    0: 5, # 极低度数节点: 采样少, 避免重复采样
    1: 10, # 低度数节点
    2: 15, # 中度数节点
    3: 30 # 高度数节点: 可采样更多邻居
}

def get_adaptive_max_neighbors(self, node):
    """根据节点度数层级获取自适应max_neighbors"""
    if not self.use_adaptive_sampling:
        return self.max_neighbors
    tier = self.dataset.get_node_degree_tier(node)
    return self.adaptive_max_neighbors.get(tier, self.max_neighbors)

```

```

# === 代码位置: care_gnn_model.py ===

def sample_2hop_neighbors(self, nodes, features, max_2hop=10):
    """为低度数节点采样2-hop邻居, 扩展感受野"""
    for i, node in enumerate(nodes):
        # 仅对低度数节点进行2-hop扩展
        tier = self.dataset.get_node_degree_tier(node)
        if tier <= 1: # 度数 <= 5 的节点
            neighbors_2hop = self.dataset.get_2hop_neighbors(node)
            if len(neighbors_2hop) > 0:

```

```

# 随机采样2-hop邻居
if len(neighbors_2hop) > max_2hop:
    sampled_2hop = np.random.choice(
        list(neighbors_2hop), max_2hop, replace=False
    )
else:
    sampled_2hop = list(neighbors_2hop)
# 填充特征...

```

效果：低度数节点从几乎无邻居信息 → 可利用 2-hop 扩展的邻居信息，显著提升这部分节点的代表质量。

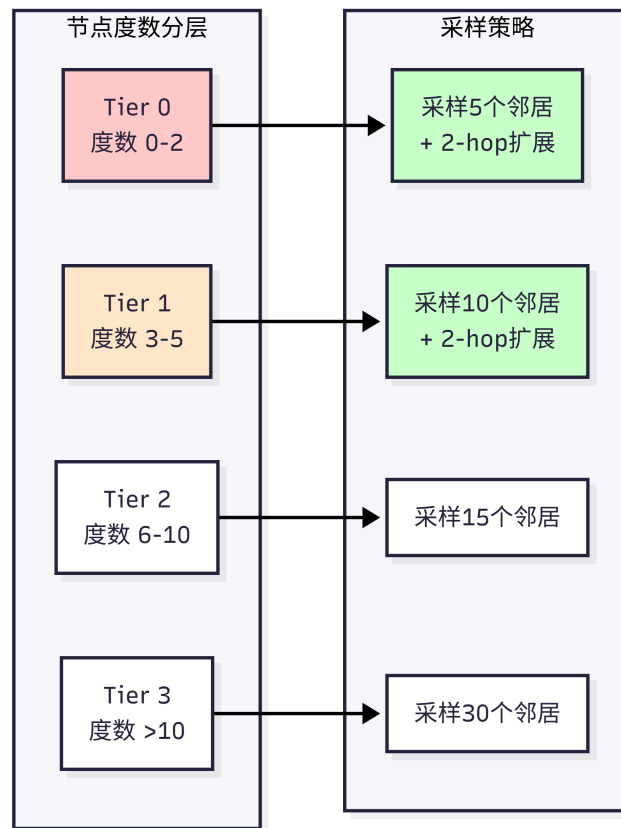


Figure 4: CARE-GNN 架构示意图

3.2 增强 2：类别不平衡处理（三重策略）

问题：欺诈样本仅占 1.18%，模型容易学到“全预测正常”的捷径。

解决方案：组合使用三种策略。

```

# === 策略1：下采样多数类 (train.py) ===
parser.add_argument('--undersample', type=int, default=8)
# 将正常类下采样到欺诈类的8倍，而非原始的84倍

# === 策略2: Focal Loss (care_gnn_model.py) ===
class FocalLoss(nn.Module):
    """聚焦难分类样本，降低易分类样本的权重"""

    def __init__(self, alpha=0.75, gamma=3.0):

```

```

super(FocalLoss, self).__init__()
self.alpha = alpha    # 正类权重
self.gamma = gamma    # 聚焦参数

def forward(self, inputs, targets):
    ce_loss = F.cross_entropy(inputs, targets, reduction='none')
    pt = torch.exp(-ce_loss)    # 预测正确的概率
    # 关键：难分类样本(pt小)权重高，易分类样本(pt大)权重小
    focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss
    return focal_loss.mean()

# === 策略3：辅助标签相似度任务 (care_gnn_model.py) ===
# 在损失函数中增加标签预测的辅助任务
total_loss = gnn_loss + lambda_1 * label_loss    # lambda_1=1.5

```

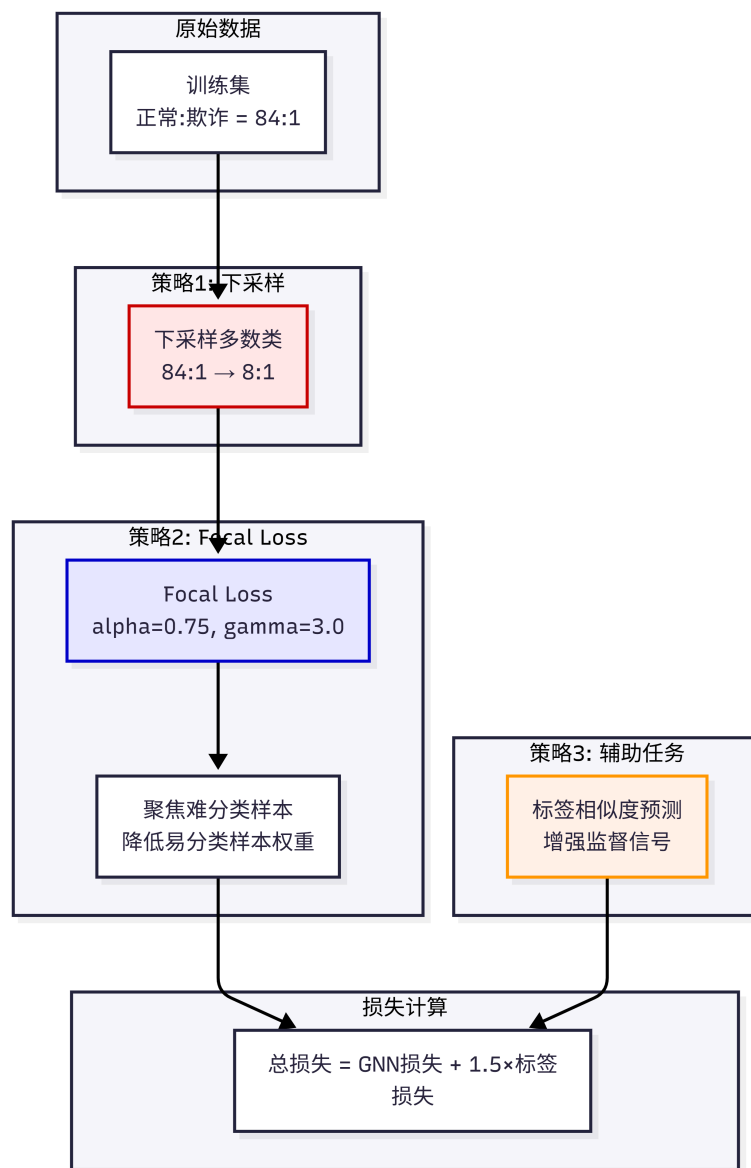


Figure 5: CARE-GNN 架构示意图

3.3 增强 3：边方向校正

发现：原始数据的边方向可能是“受害者→欺诈者”，导致 GNN 聚合的是“谁被骗了”而非“骗了谁”，这与欺诈传播的逻辑相悖。

解决方案：边方向反转。

```
# === 代码位置: data_loader.py ===

def __init__(self, data_path='phase1_gdata.npz'):
    # ...

    # 关键优化: 边方向互换 [source, target] → [target, source]
    # 原因: 原始数据可能是"受害者→欺诈者"方向
    # 反转后GNN能聚合到正确的邻居信息 (欺诈者影响的受害者)
    self.edge_index = data['edge_index'][:, [1, 0]] # 交换列!
    print(" ✓ Edge direction reversed: [source, target] → [target, source]")
```

原始方向:0.80926 **反转后:0.82563**

AUC 提升约 **1.6%**，证明边方向校正对模型理解欺诈传播模式至关重要。

4 遇到的问题与解决过程

4.1 问题 1：模型区分能力差（AUC 接近 0.5）

现象：早期版本模型训练后，验证集 AUC 始终在 0.5 左右徘徊，说明模型几乎没有学到有效的区分特征，预测接近随机猜测。

原因分析：经过排查发现主要有两方面原因：一是类别极度不平衡（正常:欺诈 = 84:1），模型倾向于忽略少数类；二是原始边方向可能与欺诈传播逻辑不符，导致 GNN 聚合的邻居信息缺乏区分性。

解决方案：

1. 引入下采样 (8:1) + Focal Loss 组合策略处理类别不平衡
2. 尝试边方向反转，使模型能聚合到更有意义的邻居信息

```
# === 代码位置: train.py ===
# 监控预测分布和AUC变化, 诊断模型学习状态
probs = val_metrics['probs']
pred_class1 = (probs[:, 1] > 0.5).sum()
print(f"Pred Class 1: {pred_class1}/{len(probs)} ({100*pred_class1/len(probs):.2f}%)")
print(f"AUC: {val_metrics['auc']:.4f}")
```

效果：通过上述调整，AUC 从约 0.5 提升到 0.80+，边方向反转后进一步提升到 0.82+。

4.2 问题 2：2-hop 邻居构建内存溢出

现象：为所有节点构建 2-hop 邻居时，内存占用超过可用内存，程序崩溃。

原因分析：数据集包含 400 万+ 节点，若为每个节点都构建完整的 2-hop 邻居集合，空间复杂度会急剧膨胀。

解决方案：分析发现真正需要 2-hop 扩展的是低度数节点（度数≤5），高度数节点 1-hop 邻居已经足够丰富。因此仅为低度数节点构建 2-hop 邻居，既保证了效果又控制了内存开销。

```
# === 代码位置: data_loader.py ===
def _build_2hop_neighbors(self):
    """为低度数节点构建2-hop邻居"""
    # 仅为低度数节点构建 (tier 0和1, 即度数<=5)
    low_degree_mask = self.degree_tiers <= 1
    low_degree_nodes = np.where(low_degree_mask)[0]
    # 而非全部 4,024,623 个节点

    for node_id in low_degree_nodes:
        neighbors_1hop = global_adj[node_id]
        neighbors_2hop = set()
        for neighbor in neighbors_1hop:
            neighbors_2hop.update(global_adj[neighbor])
        neighbors_2hop -= neighbors_1hop # 排除1-hop
        neighbors_2hop.discard(node_id) # 排除自己
        self.adj_2hop[node_id] = neighbors_2hop
```

5 最终模型配置与运行

5.1 关键超参数说明

参数	值	说明
embed_dim	128	嵌入维度，平衡表达能力与计算效率
max_neighbors	30	最大邻居采样数（高度数节点）
focal_alpha	0.75	Focal Loss 正类权重
focal_gamma	3.0	Focal Loss 聚焦参数
undersample	8	下采样比例（8:1）
use_adaptive_sampling	True	启用自适应采样
use_2hop	True	启用 2-hop 邻居扩展

Table 1: 关键超参数配置

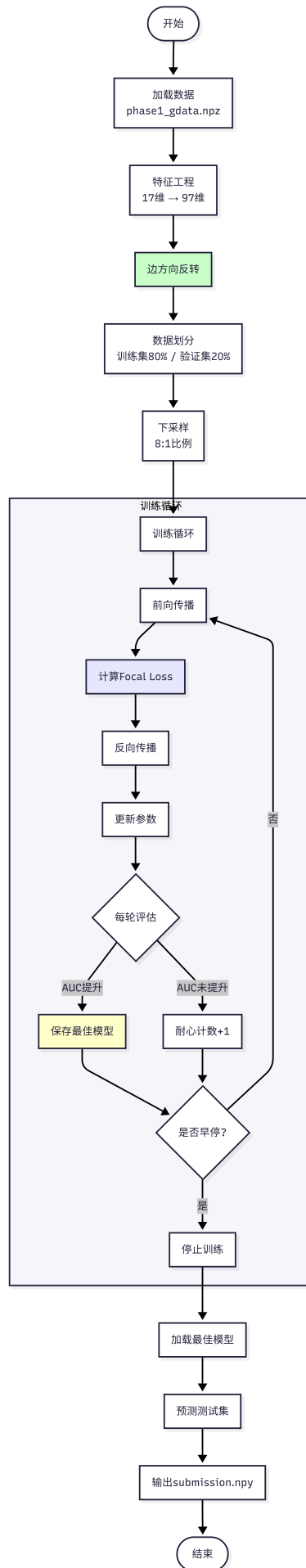


Figure 6: CARE-GNN 架构示意图

6 总结

6.1 与经典 CARE-GNN 的对比

方面	经典 CARE-GNN	本项目实现
邻居采样	固定数量	自适应采样 (按度数分层)
感受野	1-hop	1-hop + 条件性 2-hop
损失函数	交叉熵	Focal Loss + 辅助任务
边处理	原始方向	方向反转 (+1.6% AUC)
类别平衡	无特殊处理	下采样 + Focal Loss

Table 2: 与经典 CARE-GNN 的对比

6.2 核心创新点

- 1. **自适应 2-hop 扩展**: 针对金融图中大量低度数节点的特点,通过 2-hop 邻居扩展解决信息稀疏问题
- 2. **边方向校正**: 发现并修正原始数据边方向与欺诈传播逻辑的不一致,带来显著性能提升
- 3. **多策略类别平衡**: 组合下采样、Focal Loss 和辅助任务,有效应对 1.18% 的极端类别不平衡

6.3 最终成绩

测试集 A AUC: **0.82563**